Super Mario Game — Sequential Task Checklist

> **Reference**: SPEC.md v2.0 for all constants and specifications. **Rule**: Each phase = one feature branch off `dev`. Merge to `dev` when phase is complete. **Version**: 2.0 — Updated 2026-06-02 with expanded 110-feature scope.

Phase 0 — Environment Setup ✅ COMPLETE

- [x] Initialize git repo, `dev` branch
- [x] Create directory structure (`include/` , `src/` , `assets/` , `logs/` under nested app folder)
- [x] Configure CMakeLists.txt with SFML 3.0.2 + ImGui-SFML
- [x] Write baseline main.cpp with window + ImGui
- [x] Verify compilation
- [x] Create AGENTS.md, README.md, .gitignore
- [x] Commit: `feat: initialize project environment with SFML and ImGui`

Phase 1 — Core Engine (`feature/core-engine`)

> **Goal**: Game loop, state management, resource loading, input, sound, events.
> **Branch**: `git checkout -b feature/core-engine dev`

1.1 Constants & Utilities

- [x] Create Constants.hpp — all game constants from SPEC §4
- `WINDOW_WIDTH=1280` , `WINDOW_HEIGHT=720` , `TILE_SIZE=32`
- `GRAVITY=0.5f` , `MARIO_WALK_SPEED=150.f` , `MARIO_RUN_SPEED=300.f`
- `MARIO_JUMP_HEIGHT=128.f` , `FIXED_TIMESTEP=1.0f/60.0f`
- `LUIGI_JUMP_MULT=1.2f` , `LUIGI_SPEED_MULT=0.85f` , `LUIGI_GRAVITY_MULT=0.9f`
- `STAR_DURATION=10.0f` , `LEVEL_TIME=300.0f` , `INITIAL_LIVES=3`

- `COYOTE_FRAMES=6`, `JUMP_BUFFER_FRAMES=6` [v2.0]
- `WALL_SLIDE_SPEED=50.f`, `GROUND_POUND_SPEED=600.f` [v2.0]
- `COMBO_MULTIPLIERS={1,2,4,8}` [v2.0]
- [x] Create MathUtils.hpp + MathUtils.cpp
- `clamp()`, `lerp()`, `sign()`, vector helpers
- [x] Commit: `feat: add game constants and math utilities`

1.2 Resource Manager (Singleton)

- [x] Create ResourceManager.hpp + ResourceManager.cpp
- Singleton with `static ResourceManager& getInstance()`
- `loadTexture(id, path)`, `getTexture(id)` → `sf::Texture&`
- `loadFont(id, path)`, `getFont(id)` → `sf::Font&`
- `loadSoundBuffer(id, path)`, `getSoundBuffer(id)` → `sf::SoundBuffer&`
- Internal `std::unordered_map<std::string, sf::Texture>` etc.
- Delete copy/move constructors
- [x] Commit: `feat: implement ResourceManager singleton`

1.3 Sound Manager (Singleton)

- [x] Create SoundManager.hpp + SoundManager.cpp
- Singleton with `static SoundManager& getInstance()`
- `playSound(id)` — plays a sound effect (pool of `sf::Sound` objects)
- `playMusic(path)`, `stopMusic()`, `pauseMusic()`
- `setSFXVolume(float)`, `setMusicVolume(float)`
- Uses `ResourceManager` for sound buffers
- `sf::Music` for streaming BGM
- Surface-dependent footstep support [v2.0]
- [x] Commit: `feat: implement SoundManager singleton`

1.4 Event Bus (Observer Pattern)

- [x] Create EventBus.hpp + EventBus.cpp
- `enum class EventType { CoinCollected, EnemyDefeated, PlayerDied, PowerUpCollected, LevelComplete, ComboHit, AchievementUnlocked,`

`StarCoinCollected, PSwitchActivated, BossDefeated, ... }` [v2.0: expanded from 4 to 15+ types]

- `struct GameEvent { EventType type; std::any data; }`
- `subscribe(EventType, std::function<void(const GameEvent&)>)` → returns subscription ID
- `unsubscribe(subscriptionId)`
- `publish(GameEvent)`
- [x] Commit: `feat: implement EventBus observer pattern`

1.5 Input Manager (Command Pattern)

- [x] Create InputManager.hpp + InputManager.cpp
- `ICommand` interface: `virtual void execute(Character&) = 0`
- Concrete commands: `JumpCommand`, `MoveLeftCommand`, `MoveRightCommand`, `FireCommand`, `RunCommand`, `CrouchCommand`, `GroundPoundCommand`, `WallJumpCommand` [v2.0: 8+ commands]
- `InputManager` maps `sf::Keyboard::Key` → `std::unique_ptr<ICommand>`
- `handleInput(sf::Event, Character&)` — processes key events
- `update(Character&)` — processes held keys (continuous movement)
- Support Player 1 (WASD) and Player 2 (Arrow keys) mappings
- Key rebinding support with persistence [v2.0]
- [x] Commit: `feat: implement InputManager with Command pattern`

1.6 Game State Manager (State Pattern)

- [x] Create IGameState.hpp — abstract interface
- `virtual void enter() = 0`
- `virtual void exit() = 0`
- `virtual void handleInput(sf::Event) = 0`
- `virtual void update(float dt) = 0`
- `virtual void render(sf::RenderTarget&) = 0`
- `virtual ~IGameState() = default`
- [x] Create GameStateManager.hpp + GameStateManager.cpp
- Stack-based: `pushState()`, `popState()`, `changeState()`
- `std::stack<std::unique_ptr<IGameState>>`
- `getCurrentState()` → `IGameState*`

- Calls enter/exit on transitions
- Support for 9 game states [v2.0: +WorldMapState, StatisticsState]
- [x] Commit: `feat: implement GameStateManager with state stack`

1.7 Game Class (Singleton, Main Loop)

- [x] Create Game.hpp + Game.cpp
- Singleton: `static Game& getInstance()`
- Owns `sf::RenderWindow` (1280×720)
- Owns `GameStateManager`
- Fixed timestep game loop with interpolation
- `run()`, `quit()`
- ImGui integration: `ImGui::SFML::Update()` in loop
- Initial state: push `MenuState` (placeholder for now)
- [x] Refactor main.cpp — calls `Game::getInstance().run()`
- [x] Commit: `feat: implement Game singleton with fixed-timestep loop`

1.8 Placeholder States

- [x] Create MenuState.hpp + MenuState.cpp — placeholder "Press Enter to Start"
- [x] Create PlayingState.hpp + PlayingState.cpp — placeholder colored screen
- [x] Update CMakeLists.txt — add all new source files
- [x] Verify build compiles and runs with state transitions
- [x] Commit: `feat: add placeholder MenuState and PlayingState`
- [ ] **Merge**: `git checkout dev && git merge feature/core-engine`

Phase 2 — Physics Engine (`feature/physics`)

> **Goal**: AABB collision, gravity, velocity, tile-based collision resolution, spatial hashing. **Branch**: `git checkout -b feature/physics dev`

2.1 AABB

- [x] Create AABB.hpp + AABB.cpp
- `struct AABB { float x, y, width, height; }`

- `bool intersects(const AABB& other) const`
- `AABB getOverlap(const AABB& other) const`
- `bool contains(float px, float py) const`
- `sf::Vector2f getCenter() const`
- [x] Commit: `feat: implement AABB collision primitive`

2.2 Spatial Hash [v2.0]

- [x] Create SpatialHash.hpp + SpatialHash.cpp
- Grid cell size: 64×64 pixels
- `insert(Entity*, AABB)`, `query(AABB)` → `std::vector<Entity*>`
- `clear()` — called every frame before re-insertion
- O(n) average collision broadphase
- [x] Commit: `feat: implement SpatialHash for collision broadphase`

2.3 Collision Detector

- [x] Create CollisionDetector.hpp + CollisionDetector.cpp
- `struct CollisionInfo { bool collided; sf::Vector2f overlap; sf::Vector2f normal; Entity* other; }`
- `checkEntityVsEntity(Entity&, Entity&)` → `CollisionInfo`
- `checkEntityVsTileMap(Entity&, TileMap&)` → `std::vector<CollisionInfo>`
- Direction detection: top, bottom, left, right collision normals
- Uses SpatialHash for broadphase [v2.0]
- [x] Commit: `feat: implement CollisionDetector with direction sensing`

2.4 Collision Resolver

- [x] Create CollisionResolver.hpp + CollisionResolver.cpp
- `resolveEntityVsTile(Entity&, CollisionInfo&)` — push entity out of tile
- `resolveEntityVsEntity(Entity&, Entity&, CollisionInfo&)` — stomp/damage/bounce
- `resolvePlayerVsEnemy(Character&, Enemy&, CollisionInfo&)` — stomp vs side hit
- `resolvePlayerVsItem(Character&, Item&, CollisionInfo&)` — collect/activate
- Ground detection: set `onGround = true` when bottom collision with tile
- Wall detection: set `onWall = true` for wall slide/jump [v2.0]

- Surface type detection (ice, conveyor, water) [v2.0]
- [x] Commit: `feat: implement CollisionResolver with response logic`

2.5 Physics Engine

- [x] Create PhysicsEngine.hpp + PhysicsEngine.cpp
- `applyGravity(Entity&, float dt)` — with water/ice modifiers [v2.0]
- `integrateVelocity(Entity&, float dt)` — position += velocity * dt
- `update(std::vector<Entity*>&, TileMap&, float dt)`:
  1. Rebuild spatial hash
  2. Apply gravity (with zone modifiers — water, etc.)
  3. Apply surface effects (ice friction, conveyor push) [v2.0]
  4. Integrate velocities
  5. Detect collisions (broadphase via SpatialHash)
  6. Resolve collisions
  7. Update ground/wall states
- Coyote time and jump buffering logic [v2.0]
- Momentum and acceleration curves [v2.0]
- ImGui debug: toggle collision box rendering, show velocity vectors
- [x] Update CMakeLists.txt
- [x] Write a test: place a rectangle on screen, verify it falls and stops on a floor tile
- [x] Commit: `feat: implement PhysicsEngine with gravity and collision pipeline`
- [ ] **Merge**: `git checkout dev && git merge feature/physics`

Phase 3 — Entity Hierarchy (`feature/entities`)

> **Goal**: All game entities — player characters, enemies, items, blocks, and EntityFactory. **Branch**: `git checkout -b feature/entities dev`

3.1 Entity Base Class

- [x] Create Entity.hpp + Entity.cpp
- Abstract base. Members: `position`, `velocity`, `boundingBox`, `active`, `sprite`
- Pure virtual: `update(float dt)`, `render(sf::RenderTarget&)`

- Virtual: `getBoundingBox()`, `isActive()`, `destroy()`
- Virtual destructor
- [ ] Commit: `feat: implement abstract Entity base class`

3.2 Character Base Class

- [x] Create Character.hpp + Character.cpp
- Inherits `Entity`. Members: `health`, `speed`, `jumpForce`, `onGround`, `onWall`, `facingRight`
- Methods: `moveLeft()`, `moveRight()`, `jump()`, `takeDamage()`
- [ ] Commit: `feat: implement Character base class`

3.3 Player Base Class

- [x] Create Player.hpp + Player.cpp
- Inherits `Character`. Members: `lives`, `coins`, `score`
- Methods: `run()`, `wallJump()`, `groundPound()`, `crouch()`, `slide()`, `shootFireball()` [v2.0]
- `IPlayerState` pattern for 5 base forms: Small, Super, Fire, Cape, Mini [v2.0]
- Decorators for temporary forms: `StarDecorator`, `MegaDecorator` [v2.0]
- State transitions: `powerUp(ItemType)`, `powerDown()`
- Invincibility frames timer after hit
- Coyote time counter, jump buffer counter [v2.0]
- Combo counter [v2.0]
- [ ] Commit: `feat: implement Player base class with state management`

3.4 Mario

- [x] Create Mario.hpp + Mario.cpp
- Inherits `Player`
- Physics values from `Constants.hpp`: walk=150, run=300, jump=4 tiles
- `shootFireball()` — if Fire state, max 2 active
- Animation state tracking (idle, walk, run, jump, crouch, slide, wall_slide, ground_pound, swim, climb, die) [v2.0: expanded]
- [ ] Commit: `feat: implement Mario entity`

3.5 Luigi

- [x] Create Luigi.hpp + Luigi.cpp
- Inherits `Player`
- Modified physics: speed×0.85, jump×1.2, airGravity×0.9
- `doubleJump()` — can jump once more while airborne
- [ ] Commit: `feat: implement Luigi entity with double jump`

3.6 Unlockable Characters [v2.0]

- [x] Create Toad.hpp + Toad.cpp
- Inherits `Player`. Speed ×1.3, Jump ×0.8, no slide friction delay.
- [x] Create Peach.hpp + Peach.cpp
- Inherits `Player`. Float ability (hold jump to hover 1.5s), speed ×0.9.
- [ ] Commit: `feat: implement unlockable characters Toad and Peach`

3.7 Enemy Base + AI Strategies

- [x] Create IMovementStrategy.hpp — interface
- `virtual void execute(Enemy& enemy, float dt) = 0` (Template Method skeleton)
- `virtual ~IMovementStrategy() = default`
- [x] Create strategies:
- [x] PatrolStrategy.hpp/.cpp — walk, reverse on wall, fall off (or ledge-aware for variants) [v2.0]
- [x] ChaseStrategy.hpp/.cpp — idle until within 250px, move toward target
- [x] FlyStrategy.hpp/.cpp — sinusoidal vertical movement + horizontal patrol
- [x] TimerEmergenceStrategy.hpp/.cpp — emerge/retreat on timer (Piranha Plant) [v2.0]
- [x] LinearStrategy.hpp/.cpp — straight-line travel (Bullet Bill) [v2.0]
- [x] HammerThrowStrategy.hpp/.cpp — jump between platforms + arc throws [v2.0]
- [x] TetheredChaseStrategy.hpp/.cpp — lunge toward player, snap back to anchor [v2.0]
- [x] ProximityTriggerStrategy.hpp/.cpp — idle, slam, rise (Thwomp) [v2.0]
- [x] Create Enemy.hpp + Enemy.cpp
- Inherits `Character`. Holds `std::unique_ptr<IMovementStrategy>`
- `update()` delegates fully to `aiStrategy->execute()` [v2.0]
- `onStomped()`, `onHitByFireball()` — virtual
- Scoring: each enemy kill grants points (configurable via JSON) [v2.0]

- [ ] Commit: `feat: implement Enemy base class and 8 AI movement strategies`

3.8 Concrete Enemies (Original)

- [x] Create Goomba.hpp/.cpp — PatrolStrategy, variant support (brown/red) [v2.0]
- [x] Create KoopaTroopa.hpp/.cpp — PatrolStrategy, shell state, variant support [v2.0]
- [x] Create KoopaParatroopa.hpp/.cpp — FlyStrategy, variant support [v2.0]
- [x] Create Boo.hpp/.cpp — ChaseStrategy, invulnerable
- [ ] Commit: `feat: implement Goomba, KoopaTroopa, Paratroopa, Boo with variants`

3.9 Concrete Enemies (New in v2.0)

- [x] Create PiranhaPlant.hpp/.cpp — TimerEmergenceStrategy, pipe-bound
- [x] Create BulletBill.hpp/.cpp — LinearStrategy, stompable, spawned from Bill Blaster
- [x] Create HammerBro.hpp/.cpp — HammerThrowStrategy, arc projectiles
- [x] Create Thwomp.hpp/.cpp — ProximityTriggerStrategy, 3-state lifecycle
- [x] Create ChainChomp.hpp/.cpp — TetheredChaseStrategy, 4-tile radius
- [x] Create Lakitu.hpp/.cpp — FlyStrategy + spawns Spinies via Factory
- [x] Create Spiny.hpp/.cpp — PatrolStrategy, not stompable
- [ ] Commit: `feat: implement 7 new enemy types (Piranha, Bullet, Hammer, Thwomp, Chain, Lakitu, Spiny)`

3.10 Items (Original)

- [x] Create Item.hpp + Item.cpp
- Inherits `Entity`. `collected` flag. `virtual activate(Player&)`, `collect()`
- [x] Create: Mushroom.hpp/.cpp, FireFlower.hpp/.cpp, Coin.hpp/.cpp, Star.hpp/.cpp, OneUpMushroom.hpp/.cpp
- [ ] Commit: `feat: implement original 5 items (Mushroom, FireFlower, Coin, Star, 1-UP)`

3.11 Items (New in v2.0)

- [x] Create CapeFeather.hpp/.cpp — grants Cape state (glide + swoop)
- [x] Create MegaMushroom.hpp/.cpp — temporary giant (8s, Decorator pattern)
- [x] Create MiniMushroom.hpp/.cpp — half-size, walk on water
- [x] Create POWBlock.hpp/.cpp — area-of-effect via EventBus
- [x] Create PSwitch.hpp/.cpp — bricks↔coins for 15s (Command pattern)

- [x] Create Trampoline.hpp/.cpp — bounces player ~6 tiles, carriable
- [x] Create StarCoin.hpp/.cpp — 3 per level, tracked in save data
- [ ] Commit: `feat: implement 7 new items (Cape, Mega, Mini, POW, PSwitch, Trampoline, StarCoin)`

3.12 Blocks (Original)

- [x] Create Block.hpp + Block.cpp
- Inherits `Entity`. `breakable` flag. `virtual onHitFromBelow(Player&)`
- [x] Create: BrickBlock.hpp/.cpp, QuestionBlock.hpp/.cpp, Pipe.hpp/.cpp, Flagpole.hpp/.cpp
- [ ] Commit: `feat: implement original 4 blocks (Brick, Question, Pipe, Flagpole)`

3.13 Blocks (New in v2.0)

- [x] Create HiddenBlock.hpp/.cpp — invisible until hit from below
- [x] Create MovingPlatform.hpp/.cpp — path-based movement, carries player
- [x] Create FallingPlatform.hpp/.cpp — 4-state lifecycle (State pattern)
- [x] Create IceBlock.hpp/.cpp — reduced friction surface
- [x] Create ConveyorBelt.hpp/.cpp — directional push force
- [ ] Commit: `feat: implement 5 new blocks (Hidden, Moving, Falling, Ice, Conveyor)`

3.14 Entity Factory (Factory Pattern)

- [x] Create EntityFactory.hpp + EntityFactory.cpp
- `enum class EntityType { ... }` — 25+ types [v2.0]
- `static std::unique_ptr<Entity> create(EntityType type, sf::Vector2f position)`
- `static std::unique_ptr<Entity> create(EntityType type, sf::Vector2f position, const json& config)` — for level loader and variants
- Config-driven entity definitions: reads from `entities.json` [v2.0]
- [x] Update CMakeLists.txt with all entity source files
- [x] Verify build compiles
- [ ] Commit: `feat: implement EntityFactory for all 25+ entity types`
- [ ] **Merge**: `git checkout dev && git merge feature/entities`

Phase 4 — Tilemap & Levels (`feature/tilemap-levels`)

Goal: Level loading from JSON, tilemap rendering, camera scrolling, world map.
Branch: `git checkout -b feature/tilemap-levels dev`

4.1 TileMap

- [x] Create TileMap.hpp + TileMap.cpp
- 2D grid with tile IDs and tile properties (friction, conveyor direction) [v2.0]
- Tile types: `Empty=0, Ground=1, Brick=2, Question=3, Pipe=4, Ice=5, Conveyor=6, Water=7, ...` [v2.0: expanded]
- `render(sf::RenderTarget&, Camera&)` — only draw visible tiles
- `getTileAt()` , `worldToGrid()` , `gridToWorld()`
- `getTileSurfaceType()` — returns Ice, Normal, Water, Conveyor for physics [v2.0]
- P-Switch support: `swapBricksAndCoins()` [v2.0]
- Water zone tracking [v2.0]
- [ ] Commit: `feat: implement TileMap with surface types and water zones`

4.2 Level Loader

- [x] Create LevelLoader.hpp + LevelLoader.cpp
- `loadLevel(const std::string& jsonPath)` → `Level` struct
- Parse expanded JSON format from SPEC §9.6 (star coins, water zones, moving platforms, variants) [v2.0]
- Uses `EntityFactory::create()` with variant config [v2.0]
- [ ] Commit: `feat: implement LevelLoader with expanded JSON parsing`

4.3 Camera

- [x] Create Camera.hpp + Camera.cpp
- `follow()` — smooth follow with lookahead
- `setBounds()` — clamp to level edges
- Multiple scroll modes: horizontal, vertical, autoscroll [v2.0]
- Screen shake support [v2.0]
- `getVisibleBounds()` → `AABB` for culling
- [ ] Commit: `feat: implement Camera with multi-mode scrolling and screen shake`

4.4 Design Level Files

- [x] Create level_1.json — Overworld/Grassland + swimming section [v2.0]
- [x] Create level_2.json — Underground/Cave + ice blocks + Boom Boom mid-boss [v2.0]
- [x] Create level_3.json — Castle/Lava + Thwomps + autoscroll section + Bowser [v2.0]
- [x] Create bonus_1.json — Coin-filled bonus room [v2.0]
- [x] Create entities.json — Config-driven entity definitions [v2.0]
- [ ] Commit: `feat: create level files and entity config`

4.5 Integration Test

- [x] Wire `PlayingState` to load Level 1, render TileMap, spawn Mario
- [x] Verify camera follows Mario, tiles render correctly, surface types work
- [x] Update CMakeLists.txt
- [ ] Commit: `feat: integrate tilemap and level loading into PlayingState`
- [ ] **Merge**: `git checkout dev && git merge feature/tilemap-levels`

Phase 5 — Graphics & Animation (`feature/graphics`)

> **Goal**: Sprite animations, HUD, parallax backgrounds, particles, visual effects.
> **Branch**: `git checkout -b feature/graphics dev`

5.1 SpriteSheet Handler

- [ ] Create SpriteSheet.hpp/.cpp
- [ ] Commit: `feat: implement SpriteSheet texture atlas handler`

5.2 Animation System

- [ ] Create Animation.hpp/.cpp
- [ ] Create AnimationManager.hpp/.cpp
- Expanded animation states: wall_slide, ground_pound, swim, climb, crouch, slide, skid [v2.0]
- [ ] Commit: `feat: implement Animation and AnimationManager`

5.3 HUD

- [ ] Create HUD.hpp/.cpp
- Render overlay: Score, Coins, World, Time, Lives
- Combo counter display [v2.0]
- P-Switch timer bar [v2.0]
- Boss health bar [v2.0]
- Star coin indicators [v2.0]
- Floating score text [v2.0]
- [ ] Commit: `feat: implement expanded HUD with combo, boss bar, and star coins`

5.4 Minimap [v2.0]

- [ ] Create Minimap.hpp/.cpp
- 200×40 pixel overview, toggleable with M key
- Shows player (green), enemies (red), items (yellow)
- [ ] Commit: `feat: implement minimap overlay`

5.5 Parallax Background

- [ ] Implement multi-layer background rendering
- [ ] Commit: `feat: implement parallax scrolling backgrounds`

5.6 Particle System

- [ ] Create ParticleSystem.hpp/.cpp
- Object-pooled particles [v2.0]
- Types: BrickBreak, CoinSparkle, DeathPoof, Stomp, Combo, WallDust, WaterBubble, LavaEmber [v2.0: expanded]
- [ ] Commit: `feat: implement ParticleSystem with object pooling`

5.7 Screen Transitions & Screen Shake [v2.0]

- [ ] Implement fade-in/fade-out transitions
- [ ] Implement screen shake system (light/medium/heavy) [v2.0]
- [ ] Commit: `feat: implement screen transitions and shake system`

5.8 Entity Death Animations [v2.0]

- [ ] Goomba squish, enemy flip, Star kill launch, player death
- [ ] Commit: `feat: implement entity death animations`

5.9 Invincibility Visual FX [v2.0]

- [ ] Star power: rainbow color cycling + sparkle trail
- [ ] Hit invincibility: sprite flashing
- [ ] Commit: `feat: implement invincibility visual effects`

5.10 Water & Lava Animation [v2.0]

- [ ] Animated water surface (sine-wave), bubble particles
- [ ] Animated lava surface, ember particles
- [ ] Commit: `feat: implement water and lava visual effects`

5.11 Source & Integrate Assets

- [ ] Download spritesheets, wire to all entities
- [ ] Update CMakeLists.txt
- [ ] Commit: `feat: integrate sprite assets and wire to all entities`
- [ ] **Merge**: `git checkout dev && git merge feature/graphics`

Phase 6 — Audio (`feature/audio`)

Goal: Wire SoundManager, load SFX and BGM, volume controls, dynamic music.
Branch: `git checkout -b feature/audio dev`

6.1 Source Audio Assets

- [ ] Find retro SFX and BGM assets (17+ SFX, 10+ BGM tracks) [v2.0: expanded]
- [ ] Commit: `feat: add audio assets`

6.2 Wire Sound Events

- [ ] Subscribe SoundManager to EventBus (15+ event types) [v2.0: expanded]
- [ ] Surface-dependent footstep sounds [v2.0]

- [ ] Combo SFX escalation [v2.0]
- [ ] Dynamic music layer system [v2.0]
- [ ] Commit: `feat: wire all sound events with dynamic music`

6.3 Volume Controls

- [ ] Volume sliders in Options, persist to config.json
- [ ] Commit: `feat: add volume controls`
- [ ] **Merge**: `git checkout dev && git merge feature/audio`

Phase 7 — Game States & UI (`feature/game-states`)

Goal: All menu screens, world map, pause, game over, victory, statistics, achievements. **Branch**: `git checkout -b feature/game-states dev`

7.1 Animated Menu State [v2.0]

- [ ] Rewrite `MenuState` with animated background, running Mario, spinning coins
- [ ] Attract mode (demo playback after 30s idle) [v2.0]
- [ ] Options: New Game, Load Game, Options, High Scores, Statistics, Achievements, Quit [v2.0: expanded]
- [ ] Commit: `feat: implement animated MenuState`

7.2 Character Select State

- [ ] Create CharSelectState — show Mario, Luigi side-by-side
- [ ] Show Toad, Peach as locked/unlocked based on progress [v2.0]
- [ ] Commit: `feat: implement CharSelectState with unlockable display`

7.3 World Map State [v2.0]

- [ ] Create WorldMapState — overhead map with level nodes
- [ ] Animated paths, star coin display, completion icons
- [ ] Sequential level unlock
- [ ] Commit: `feat: implement WorldMapState`

7.4 Playing State (Full Implementation)

- [ ] Integrate: level load, player spawn, physics, camera, HUD, minimap
- [ ] All collision callbacks including new block/item types [v2.0]
- [ ] Swimming, climbing, wall sliding, ground pounding [v2.0]
- [ ] P-Switch timer, combo system, star coin tracking [v2.0]
- [ ] ImGui development panel
- [ ] Commit: `feat: implement full PlayingState with all v2.0 systems`

7.5 Pause State

- [ ] Create PauseState — transparent overlay with Resume, Restart, Save, World Map, Menu, Quit [v2.0: expanded options]
- [ ] Commit: `feat: implement PauseState overlay`

7.6 Game Over State

- [ ] Create GameOverState with death counter and retry screen [v2.0]
- [ ] Commit: `feat: implement GameOverState with death counter`

7.7 Victory State

- [ ] Create VictoryState — score calculations, star coin summary, level transition
- [ ] Commit: `feat: implement VictoryState`

7.8 Options State & High Scores

- [ ] Create OptionsState — volume sliders, difficulty selection, controls rebinding [v2.0]
- [ ] Colorblind mode toggle [v2.0]
- [ ] High score display and persistence
- [ ] Commit: `feat: implement OptionsState with difficulty and key rebinding`

7.9 Statistics State [v2.0]

- [ ] Create StatisticsState — total enemies, coins, deaths, time, combos
- [ ] Commit: `feat: implement StatisticsState`

7.10 Achievements Display [v2.0]

- [ ] Achievement list screen (from Main Menu)

- [ ] Toast notification system (top-right slide-in)
- [ ] Commit: `feat: implement achievement display and toast notifications`
- [ ] Update CMakeLists.txt
- [ ] **Merge**: `git checkout dev && git merge feature/game-states`

Phase 8 — Save/Load & Persistence (`feature/save-load`)

> **Goal**: JSON serialization, save slots, auto-save, statistics, achievements, config persistence. **Branch**: `git checkout -b feature/save-load dev`

8.1 Serializer

- [x] Create Serializer.hpp/.cpp — save/load slots using SPEC §12.2 schema [v2.0: expanded with progress, stats, achievements, settings]
- [ ] Commit: `feat: implement Serializer with expanded schema`

8.2 Achievement Manager [v2.0]

- [x] Create AchievementManager.hpp/.cpp — subscribes to EventBus, checks conditions, fires AchievementUnlocked
- [ ] Commit: `feat: implement AchievementManager`

8.3 Statistics Tracker [v2.0]

- [x] Create StatisticsTracker.hpp/.cpp — subscribes to EventBus, accumulates stats
- [ ] Commit: `feat: implement StatisticsTracker`

8.4 Save/Load Integration

- [x] Auto-save at checkpoints, manual save from pause menu
- [x] Load slot previews with character, level, score, star coins, play time [v2.0: enhanced preview]
- [x] Settings persistence (volume, difficulty, key bindings, colorblind mode) [v2.0]
- [ ] Commit: `feat: integrate save/load with achievements, stats, and settings`
- [ ] **Merge**: `git checkout dev && git merge feature/save-load`

Phase 9 — Enemy AI & Bosses (`feature/enemy-ai`)

Goal: Tune all enemy behaviors, implement bosses. **Branch**: `git checkout -b feature/enemy-ai dev`

9.1 AI Behavior Tuning

- [ ] Tune all 13 enemy types + 3 color variants [v2.0: expanded from 5]
- [ ] Lakitu spawner logic (Factory pattern integration) [v2.0]
- [ ] Thwomp state machine (Idle → Slam → Rise) [v2.0]
- [ ] Chain Chomp tether physics [v2.0]
- [ ] ImGui AI debug overlay
- [ ] Commit: `feat: tune all enemy AI behaviors`

9.2 Boom Boom Mid-Boss [v2.0]

- [ ] 3-phase boss for Level 2
- [ ] Charge → spin → recover cycle
- [ ] Boss arena with locked doors
- [ ] Commit: `feat: implement Boom Boom mid-boss`

9.3 Bowser Boss

- [ ] Create Bowser — Phase 1 (walk + fire), Phase 2 (jump + faster fire)
- [ ] Bowser health bar, boss fireballs, Level 3 arena
- [ ] Commit: `feat: implement Bowser boss fight`

9.4 Difficulty Scaling [v2.0]

- [ ] DifficultyStrategy: Easy/Normal/Hard modifiers applied to all levels
- [ ] Balance entity distributions across all levels
- [ ] Commit: `feat: implement difficulty modes and level balancing`
- [ ] **Merge**: `git checkout dev && git merge feature/enemy-ai`

Phase 10 — Advanced Systems (`feature/advanced-systems`) [v2.0]

Goal: Object pooling, config-driven entities, replay system, debug console.
Branch: `git checkout -b feature/advanced-systems dev`

10.1 Object Pool

- [ ] Create ObjectPool.hpp — template class for pre-allocated entity recycling
- [ ] Wire to: fireballs, particles, Bullet Bills, floating text
- [ ] Commit: `feat: implement ObjectPool template and wire to entities`

10.2 Config-Driven Entities

- [ ] Create entities.json with all entity properties
- [ ] Update EntityFactory to read from config
- [ ] Commit: `feat: implement config-driven entity definitions`

10.3 Replay System

- [ ] Create ReplayRecorder.hpp/.cpp
- Record input commands per frame
- Save/load replay files
- Deterministic playback
- [ ] Commit: `feat: implement replay recording and playback`

10.4 Debug Console

- [ ] Create DebugConsole.hpp/.cpp
- Toggle with ~, text→command parsing, autocomplete
- [ ] Commit: `feat: implement debug console with command parsing`
- [ ] **Merge**: `git checkout dev && git merge feature/advanced-systems`

Phase 11 — Two-Player & Polish (`feature/polish`)

Goal: 2P versus mode, bug fixes, edge cases, visual/audio polish. **Branch**: `git checkout -b feature/polish dev`

11.1 Two-Player Versus Mode

- [ ] Configure Player 2 keyboard mappings
- [ ] Shared camera following leading player, versus rules
- [ ] Commit: `feat: implement two-player versus mode`

11.2 Edge Cases & Bug Fixes

- [ ] Respawn at checkpoint with death animation
- [ ] Flashing invincibility frames
- [ ] Clamp player inside camera
- [ ] Shell collision chains, time-out death, 100 coin 1-UP, pipe transitions
- [ ] Commit: `feat: fix edge cases and polish`

11.3 Meta-Game [v2.0]

- [ ] New Game+ mode (mirrored levels, faster enemies)
- [ ] Daily Challenge (date-seeded procedural level)
- [ ] Unlockable character conditions
- [ ] Commit: `feat: implement New Game+, Daily Challenge, and unlockables`

11.4 Accessibility [v2.0]

- [ ] Colorblind mode (shader/palette swap)
- [ ] Audio navigation cues for menus
- [ ] Commit: `feat: implement accessibility features`

11.5 Final Testing

- [ ] Complete full walkthrough of all 3 levels + bonus rooms
- [ ] Test all power-up states, enemy types, block types
- [ ] Test save/load, statistics, achievements
- [ ] Test 2P mode, difficulty modes, key rebinding
- [ ] Verify steady 60fps
- [ ] Commit: `feat: final playtest and performance verification`
- [ ] **Merge**: `git checkout dev && git merge feature/polish`

Bonus Phases (Post-MVP)

Bonus A — Level Editor (`feature/level-editor`)

- [x] ImGui editor overlay (F1 toggle)
- [x] Tile palette, entity palette, drag-and-drop
- [x] Undo/Redo with Command Pattern
- [x] Export/import level JSON
- [x] Play-test button

Bonus B — Time Rewind (`feature/time-rewind`)

- [ ] Circular buffer storing 300 frames of game state snapshots
- [ ] Memento Pattern: `GameSnapshot` struct
- [ ] Hold Shift → reverse playback
- [ ] VHS rewind visual overlay

Bonus C — Shadow Mario (`feature/shadow-mario`)

- [ ] Record player input, spawn Shadow Mario, replay with 3s delay
- [ ] Dark/translucent sprite

Bonus D — Dynamic Lighting (`feature/dynamic-lighting`)

- [ ] GLSL fragment shader for radial light
- [ ] Underground darkness, fireball illumination
- [ ] Optional day/night cycle

Bonus E — Procedural Generation (`feature/procedural-levels`)

- [ ] "Endless Mode" menu option
- [ ] Chunk-based terrain generation
- [ ] Difficulty scaling, validated placement